

LIBRARY MANAGEMENT SYSTEM

LibraryMS

Final Project Report

Course: CS3009 — Software Engineering

Team Name: Stack Architects

Team Members:

Ali Bin Salman (22i-0894) | Muhammad Rohaan (22i-2327) | Muhammad Atyab (22i-1191)

Submission Date: April 29, 2026

1. Introduction

The Library Management System (LibraryMS) is a full-stack, web-based application developed to digitize and streamline the operations of a university library. Traditional manual library systems are inefficient, time-consuming, and prone to human error. LibraryMS replaces these manual processes with an automated, role-based digital platform that covers every stage of the library lifecycle.

The system is built using Java 17 and Spring Boot 3.5.x, with a Spring MVC and Thymeleaf front end, Spring Security for session-based authentication, Spring Data JPA/Hibernate for persistence, and MySQL 8 as the database engine. The user interface is responsive and powered by vanilla JavaScript and a shared CSS stylesheet.

Development was carried out using the Scrum agile methodology across three sprints, each delivering a distinct layer of functionality that collectively produced a complete, integrated library management solution.

1.1 Project Technology Stack

- Java 17
- Spring Boot 3.5.x
- Spring MVC + Thymeleaf
- Spring Security (session-based authentication)
- Spring Data JPA / Hibernate
- MySQL 8
- Vanilla JavaScript + shared CSS (app.css)

- Maven 3.9+

1.2 Application Access

Default server port: 8081. Route families:

- Authentication: /login, /register, /logout
- Admin portal: /admin/...
- Librarian portal: /librarian/...
- Student portal: /student/...
- Book JSON API: /api/books

2. Project Vision

The vision of LibraryMS is to create a secure, efficient, and user-friendly platform that automates all major library operations. At its core, the system aims to:

- Replace manual record-keeping with a centralized digital system.
- Improve efficiency in issuing, returning, and renewing books.
- Provide real-time availability tracking of books across the catalog.
- Automate fine calculation, management, payment tracking, and receipts.
- Deliver transparency and oversight through administrative reports.
- Ensure scalability for future feature enhancements.

The long-term goal is a reliable, role-based system that improves operational efficiency while maintaining strict access control and data integrity. As the project progressed through three sprints, the vision was incrementally realized — from foundational account management in Sprint 1, through inventory and borrowing workflows in Sprint 2, to the complete library lifecycle including reservations, fines, renewals, and administrative reporting in Sprint 3.

3. Stakeholders and Intended Use

The primary stakeholders of the Library Management System are Students, Librarians, System Administrators, and University/Institution Management. Each group interacts with the system through a dedicated role-based portal.

3.1 Students (Library Members)

- Create an account (subject to admin approval).

- Search and view available books by title, author, category, or ISBN.
- Submit borrow requests and track approval status.
- View active loans, due dates, and borrowing history.
- Renew (reissue) eligible loans with clear rule-based eligibility messages.
- Reserve unavailable books and track queue position.
- View and pay personal fines; view fine receipts.

3.2 Librarians

- Manage the book catalog — add, update, and delete books with inventory consistency.
- Approve or reject student borrowing requests.
- Record book returns and process undo-return actions.
- Manage reservation queues — view, fulfill, cancel, and expire reservations.
- Manage fines — mark as paid, waive with notes, and print receipts.

3.3 System Administrator

- Approve or reject student account creation and deletion requests.
- Manage all user accounts and roles.
- Monitor library activity through four admin reports: Fines, Activity, Issued Books, and Overdue.

3.4 University / Institution Management

- Monitor overall library performance through admin-generated reports.
- Review borrowing activity statistics and fine collection data.

4. System Features and Functionality

LibraryMS is organized into several functional modules, implemented incrementally across three Scrum sprints. All modules are live in the final integrated system.

4.1 Account Management (Sprint 1)

- Student self-registration with admin approval workflow.
- Admin approval or rejection of account creation and deletion requests.
- Role-based login system (Admin, Librarian, Student).
- Session-based authentication and logout via Spring Security.
- Autofill hardening on authentication forms.

4.2 Book Catalog Management (Sprint 2)

- Add, update, and delete books with inventory consistency checks.
- Book deletion blocked when any copy is actively borrowed.
- Auto-generated base ISBN and per-copy unique ISBNs.
- Real-time available-copy count maintained across all workflows.

4.3 Search and Viewing (Sprint 2)

- Search books by title, author, category, or ISBN.
- View book availability status and copy count.
- JSON book API (/api/books) for list and search integrations.

4.4 Borrowing Workflow (Sprint 2)

- Student submits borrow request with preferred duration.
- Librarian approves or rejects pending requests.
- System records issue date and due date upon approval.
- Book inventory decremented automatically on approval.
- Active-loan limit enforced to ensure fair usage.

4.5 Book Return and Undo Return (Sprint 3)

- Librarian records return of any active loan.
- Inventory incremented and loan marked closed on return.
- Undo return action available with business-rule validation.
- Undo return blocked when it would violate maximum active-loan constraints.

4.6 Automatic Fine Calculation (Sprint 3)

- System calculates overdue fines automatically on return.
- Configurable per-day fine rate (default: PKR 10.00/day).
- Fine linked to student account and borrow record.

4.7 Fine Management (Sprint 3)

- Librarian views unpaid and all historical fines.
- Librarian marks fines as paid or waives with notes.
- Waived adjustments and net amounts separately tracked.
- Receipt generation for students and librarians.

- Admin fine report with paid/waived/unpaid breakdowns.

4.8 Loan Renewal / Reissue (Sprint 3)

- Student renews eligible active loans for 7 or 14 days.
- Renewal blocked if loan is overdue.
- Renewal blocked if maximum renewal limit is reached.
- Clear, rule-specific block messages displayed to student.
- Unpaid fines alone do not block renewal (per current policy).

4.9 Reservation and Waitlist System (Sprint 3)

- Students reserve books with zero available copies.
- FIFO reservation queue with unique positions per book.
- Reservation status lifecycle: PENDING → READY → FULFILLED / CANCELLED / EXPIRED.
- Automatic promotion to READY when a copy becomes available.
- READY pickup window capped at 4 days (96 hours); automatic expiry if not collected.
- Students may cancel their own reservations.
- Librarians fulfill, cancel, or view all reservations including expired ones.
- Queue resequencing with pessimistic locking to prevent duplicate positions.

4.10 Administrative Reports (Sprint 3)

- Fines report (unpaid / paid / waived with net-aware totals).
- Activity report (active, completed, overdue, total loans).
- Issued books report.
- Overdue books report.
- Live report updates for near real-time monitoring.

4.11 Live Updates and UX (Sprint 3)

- 1-second polling interval on dynamic pages for near real-time updates.
- Refresh pauses automatically when user has a focused control, active text selection, or a recent alert dialog.
- Alerts remain visible for at least 5 seconds.
- Responsive layout for mobile, tablet, and desktop viewports.

5. User Stories

User stories were collected during requirements analysis and distributed across the three sprints. Each story follows the format: As a [role], I want [capability], So that [benefit].

5.1 Sprint 1 — Account Management

- US-01 — Student Account Creation** | Importance: High | Estimate: 3h
- US-02 — Approve/Reject Account Creation** | Importance: High | Estimate: 2h
- US-03 — Account Deletion Request** | Importance: Medium | Estimate: 2h
- US-04 — Approve/Reject Account Deletion** | Importance: Medium | Estimate: 2h

5.2 Sprint 2 — Catalog, Search, and Borrowing

- US-05 — Add New Book** | Importance: High | Estimate: 3h
- US-06 — Update Book Details** | Importance: High | Estimate: 2h
- US-07 — Delete Book** | Importance: Medium | Estimate: 2h
- US-08 — Maintain Book Quantity** | Importance: High | Estimate: 2h
- US-09 — Search Books** | Importance: High | Estimate: 3h
- US-10 — View Book Details** | Importance: High | Estimate: 2h
- US-11 — Request Book Borrowing** | Importance: High | Estimate: 3h
- US-12 — Approve/Reject Borrowing Request** | Importance: High | Estimate: 3h
- US-13 — Reissue Book** | Importance: Medium | Estimate: 2h

5.3 Sprint 3 — Returns, Fines, Renewals, Reservations, and Reports

- US-21 — Record Book Return** | Importance: High | Estimate: 4h
- US-22 — Automatic Fine Calculation** | Importance: High | Estimate: 5h
- US-23 — Manage Student Fines** | Importance: High | Estimate: 5h
- US-24 — Renew / Reissue Book** | Importance: High | Estimate: 6h
- US-25 — Reserve Unavailable Book** | Importance: High | Estimate: 6h
- US-26 — Manage Reservation Queue** | Importance: High | Estimate: 5h
- US-27 — Generate Admin Reports** | Importance: High | Estimate: 6h
- US-28 — Final Integration and Polish** | Importance: High | Estimate: 8h

6. Key Business Rules

The following business rules govern system behavior and were implemented across the service layer:

6.1 Book and Inventory Rules

- Book deletion is allowed only when no copy is actively borrowed.
- Deleting a book also removes related pending borrow requests, active reservations, and relevant history entries.
- Available copy count is always kept synchronized with borrow, return, and reservation actions.

6.2 Borrowing Rules

- A student may not exceed the maximum number of simultaneously active loans.
- A book request is rejected if it would breach this limit.
- Due date = approval date + requested duration.

6.3 Renewal Rules

- Renewals are available for active, non-overdue loans with remaining renewal quota.
- Overdue loans cannot be renewed.
- Unpaid fines alone do not block renewal (current policy).
- A loan cannot be renewed if another student holds an active reservation for that book.

6.4 Fine Rules

- Fine amount = overdue days × per-day rate (default: 10.00 per day).
- Fine status transitions: UNPAID → PAID or WAIVED.
- Waived amount is recorded separately so net totals are always accurate.
- Reports use net-aware calculations for paid and waived fines.

6.5 Reservation Rules

- Reservations are only accepted when zero copies of a book are available.
- A student may not hold more than one active reservation for the same book.
- Queue positions are unique per book; pessimistic locking prevents duplicates.
- Reservation statuses: PENDING → READY → FULFILLED / CANCELLED / EXPIRED.
- A READY reservation's pickup window is capped at 96 hours (4 days).
- Expired pickup windows are detected and acted on by a scheduled background job.

7. Non-Functional Requirements

NFR Category	Requirement
Performance	System shall respond to user requests within 2 seconds for search operations. Database queries shall be optimized for efficient retrieval.
Security	Only authenticated users may access the system. Role-based access control restricts unauthorized actions. Passwords are securely hashed. Autofill hardening applied to authentication forms.
Usability	The interface shall be simple, intuitive, and navigable. Responsive design supports mobile, tablet, and desktop viewports. Alerts remain visible for at least 5 seconds.
Reliability	The system shall prevent duplicate issuing or double-returning of books. Transactional service methods and pessimistic locking protect critical workflows. Data integrity is maintained at all times.
Scalability	The system shall support increasing numbers of users and records. Database structure supports future feature expansion.
Maintainability	Code follows object-oriented and layered architecture principles. Schema managed via canonical SQL file (docs/library_db_schema.sql). Migration services handle startup adjustments safely.
Availability	The system shall be operational during library working hours. Live updates and scheduled expiry jobs run continuously while the server is active.

8. Sprint Summaries

Development followed the Scrum agile methodology. Each sprint was planned on a Trello board and tracked using a burn-down chart. The Scrum Master (Muhammad Rohaan) conducted daily stand-ups and maintained sprint health throughout all three iterations.

8.1 Sprint 1 — Account Management Foundation

Duration: ~2 weeks

Focus: User registration, admin approval, and role-based login

Sprint 1 established the foundational layer of the system. The team implemented student self-registration, admin approval and rejection workflows, role-based login, and account deletion requests. Spring Security was configured for session-based authentication with role-based route protection.

Sprint 1 Deliverables:

- Student registration form with pending-approval state.
- Admin dashboard for reviewing, approving, and rejecting accounts.
- Secure login with role routing (Admin / Librarian / Student dashboards).
- Account deletion request and admin approval flow.

Create account

Register as a student, librarian, or administrator. New accounts require approval before you can sign in.

YOUR USER ID

User ID

VFQ7CE

ACCOUNT TYPE

I am registering as

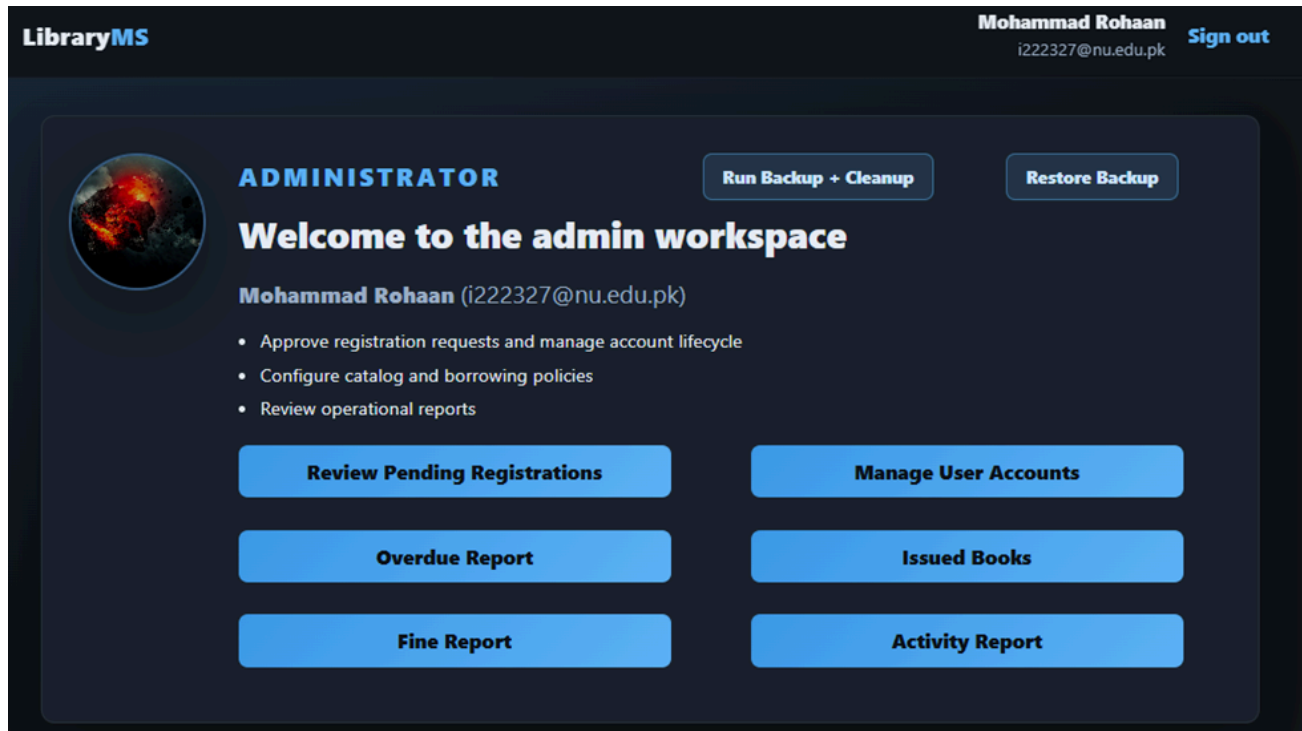
ADMIN

PROFILE

Full name

Email

Fig 8.1a — Student Registration and Admin Approval Screen



8.2 Sprint 2 — Catalog, Search, and Borrowing Workflow

Duration: ~2 weeks

Focus: Book management, search, and the full borrow request lifecycle

Building on the Sprint 1 authentication foundation, Sprint 2 introduced the core operational modules. Librarians gained the ability to manage the book catalog, and students could search for books, submit borrow requests, and track their loans. The team implemented due-date assignment, borrow-limit enforcement, and inventory synchronization.

Sprint 2 Deliverables:

- Book catalog: add, update, delete with inventory consistency.
- Book search by title, author, category, and ISBN.
- Borrow request submission by students.
- Librarian borrow approval/rejection with due-date calculation.
- Active-loan limit enforcement.
- Borrow history visible to students.

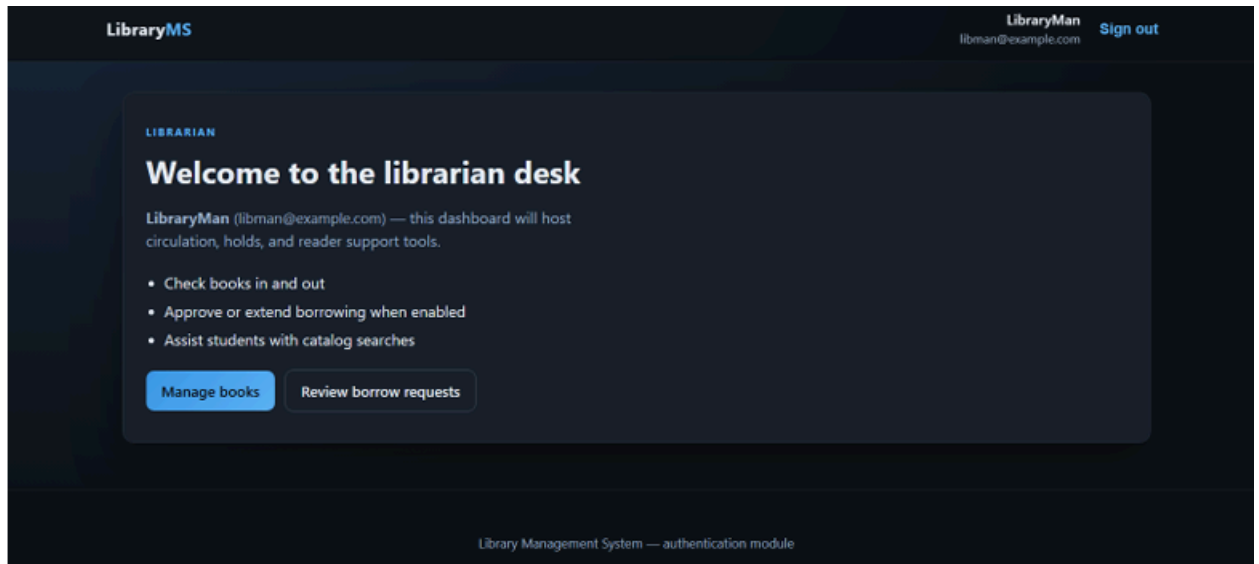


Fig 8.2a — Book Management Interface (Librarian)

Librarian / Book management

CATALOG

Book management

Create, update, and remove titles while keeping inventory counts consistent for borrowing.

Title	Author
Category	Total copies
	1

Required. Base ISBN and per-copy unique ISBN codes are auto-generated from title + category + copy number.

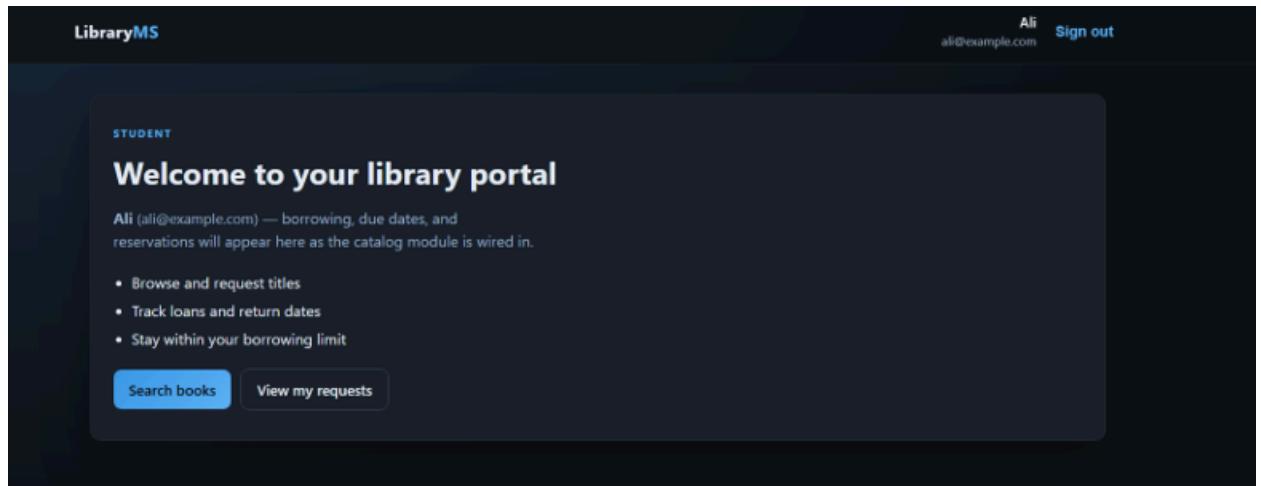


Fig 8.2b — Book Search Page (Student Portal)

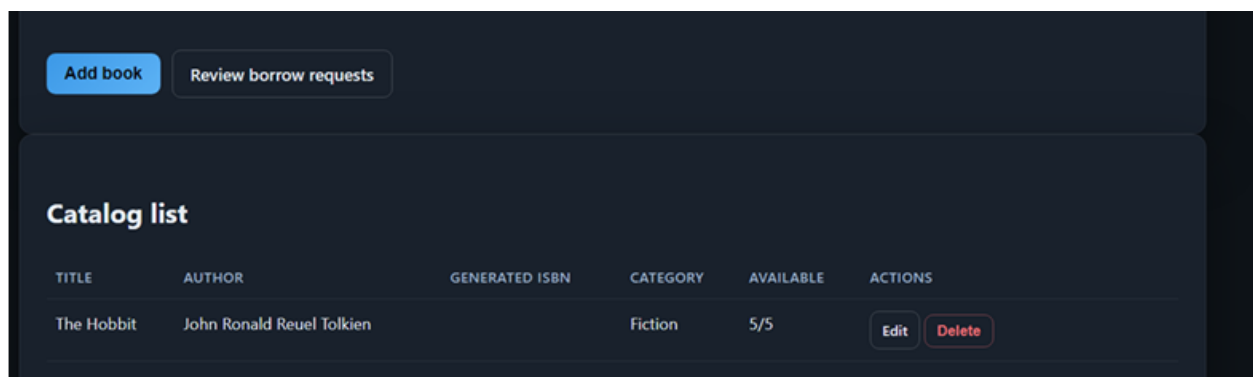
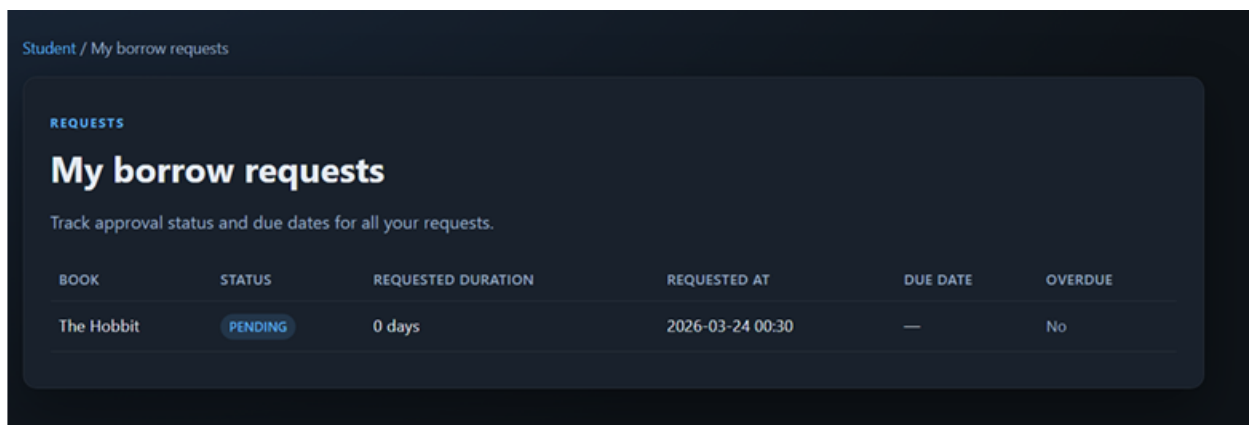


Fig 8.2c — Borrow Request Submission



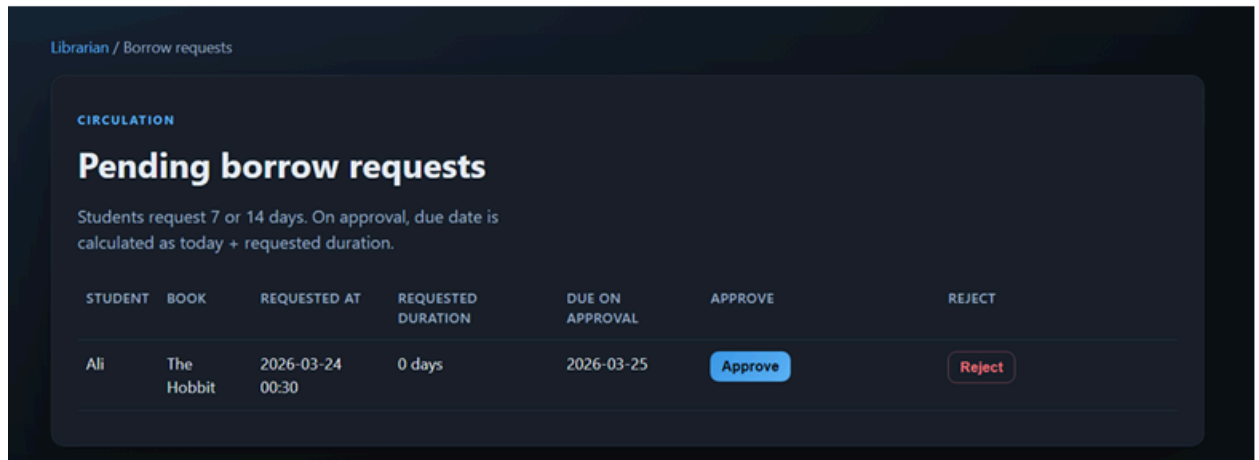


Fig 8.2d — Borrow Approval Dashboard (Librarian)

8.3 Sprint 3 — Final Operations, Fines, Reservations, and Reports

Duration: 28 days (Final Sprint)

Focus: Complete library lifecycle — returns, fines, renewals, reservations, admin reports

Sprint 3 was the final and most feature-dense sprint. It delivered the remaining advanced modules necessary to complete the full library workflow. The sprint also included final integration testing, UI polish, regression testing of Sprint 1 and Sprint 2 features, and all bug fixes.

Sprint 3 Deliverables:

- Book return processing and undo-return with rule validation.
- Automatic overdue fine calculation on return.
- Fine management for librarians (paid/waived/receipts).
- Student fine view and receipt access.
- Loan renewal with comprehensive rule enforcement.
- Reservation/waitlist system with FIFO queue, auto-expiry, and status lifecycle.
- Admin reports: Fines, Activity, Issued Books, Overdue.
- Live polling updates across all portals.
- Responsive design improvements.
- Schema migration service for safe startup adjustments.

Burn-Down Chart: Sprint 3 started with 50 task points. Progress was slightly behind plan in the first half due to complex integrations across borrow records, reservations, and fine logic. The team caught up in the second half, completing all tasks by Day 28.

LibraryMS

Bilal
022328@nu.edu.pkSign out

Librarian / Fines

Receipt Printing (Unpaid Fines Only)

Receipts are available only while the fine is unpaid.

RECEIPT ID	STUDENT	UNPAID LINES	TOTAL FINE	WAIVED ADJUSTMENT	NET AMOUNT	STATUS	ACTIONS
966046635	Yaqoob 4QRTDX	DevOps L8J64	3,000.00 PKR	2500.00 Apply	500.00 PKR	UNPAID	Print receipt

Live Overdue Books

These books are still out and past their due date. Charges show up only after the book is brought back. If a charge was added by mistake after a return, you can reverse it with Undo return on Active loans.

[Go to Active loans](#)

STUDENT	BOOK	DUE DATE	DAYS OVERDUE	STATUS	PER-DAY FINE	UNPAID FINE	TOTAL FINE
Yaqoob 4QRTDX	Spoken English 8896WH	2026-04-15	14	OVERDUE	50 PKR/day	700.00 PKR	700.00 PKR

Live Overdue Books

These books are still out and past their due date. Charges show up only after the book is brought back. If a charge was added by mistake after a return, you can reverse it with Undo return on Active loans.

[Go to Active loans](#)

STUDENT	BOOK	DUE DATE	DAYS OVERDUE	STATUS	PER-DAY FINE	UNPAID FINE	TOTAL FINE
Yaqoob 4QRTDX	Spoken English 8896WH	2026-04-15	14	OVERDUE	50 PKR/day	700.00 PKR	700.00 PKR

Fine Status Management

Update status and add one optional note.

[Unpaid](#)[Paid](#)[Waived](#)[All](#)

STUDENT	BOOK	TOTAL FINE	WAIVED ADJUSTMENT	NET FINE	STATUS	NOTES	UPDATE
Yaqoob (4QRTDX)	DevOps	3,000.00 PKR	2,500.00 PKR	500.00 PKR	UNPAID	—	Unpaid Optional note Save

Active Loans

All books currently borrowed and not yet returned. Overdue loans are highlighted. Use the Return action to process a return — fines are calculated automatically.

STUDENT	BOOK	COPY ISBN	ISSUED	DUE DATE	STATUS	ACTION
Yaqoob ID: 4QRTDX	Spoken English Mangler	SPOKEN-LITE-BH3ABC-C001	2026-03-25	2026-04-15	OVERDUE	<button>Return</button>
Yaqoob ID: 4QRTDX	Computer Networks V2 Mikle Aradan MK	COMPUT-INFO-FD45B2-C002	2026-04-29	2026-05-27	ACTIVE	<button>Return</button>
Ahmed ID: DNZCTG	Object Oriented Programming Hallen Walk	OBJECT-PROG-905C13-C001	2026-04-29	2026-05-27	ACTIVE	<button>Return</button>
Ahmed ID: DNZCTG	Computer Networks V2 Mikle Aradan MK	COMPUT-INFO-FD45B2-C001	2026-04-29	2026-05-27	ACTIVE	<button>Return</button>

[View Fines](#)

WAITLIST

Reservation Queue

Manage reservations in first-come, first-served order. READY reservations are expired automatically if not picked up within 4 days.

[Ready Only](#) [All Reservations](#) [Expired Only](#)

STUDENT	BOOK	QUEUE #	REQUESTED DAYS	STATUS	RESERVED	PICKUP DEADLINE	ACTIONS
Ahmed 1222338@nu.edu.pk	DevOpps Kalen Alex	2	28 days	CANCELLED	2026-03-29	2026-05-01 06:00	—
Yaqoob 1222332@nu.edu.pk	Machine Learning For Robotics David Kelson	8	28 days	CANCELLED	2026-03-29	—	—
Yaqoob 1222332@nu.edu.pk	Machine Learning For Robotics David Kelson	1	28 days	EXPIRED	2026-03-29	—	—
Ahmed	Machine Learning For	1	28 days	CANCELLED	2026-03-29	—	—

Student / My Loans

CIRCULATION

My Active Loans

You can renew a loan up to 3 times. After 3 renewals, return the book and place a new borrow request; if unavailable, reserve it.

BOOK	COPY ISBN	ISSUED	DUE DATE	STATUS	RENEWALS	RENEW
Object Oriented Programming Hallen Walk	OBJECT-PROG-905C13-C001	2026-04-29	2026-05-27	ACTIVE	0 / 3	7 days <button>Renew</button>
Computer Networks V2 Mikle Aradan MK	COMPUT-INFO-FD45B2-C001	2026-04-29	2026-05-27	ACTIVE	1 / 3 Original due: 2026-05-20	7 days <button>Renew</button>

[My Borrow Requests](#) [My Fines](#)

LibraryMS

Ahmed
i222330@nu.edu.pkSign out

Student / My Fines

ACCOUNT

My Fines

Fines are issued when books are returned after the due date.
Visit the library to pay outstanding fines.

All fines have been settled. Great job keeping up with returns!

BOOK	DUE DATE	DAYS LATE	AMOUNT (PKR)	STATUS	ISSUED	NOTES
Machine Learning For Robotics	2026-04-15	13	PKR 6,500.00	PAID	2026-04-29	—
Spoken English	2026-04-22	6	PKR 300.00	PAID	2026-04-29	—

— My Borrow Requests

LibraryMS

Ahmed
i222330@nu.edu.pkSign out

My Reservations

When a book has no available copies, you can reserve it. You'll be notified when a copy becomes available — pick it up within the stated window.

BOOK	REQUESTED DAYS	QUEUE POSITION	STATUS	RESERVED ON	PICKUP DEADLINE	ACTIONS
Machine Learning For Robotics David Kelson	28 days	—	CANCELLED	2026-04-29	—	—
Machine Learning For Robotics David Kelson	28 days	—	CANCELLED	2026-04-29	—	—
DevOps Kalen Alex	28 days	—	FULFILLED	2026-04-29	2026-05-01 03:35	—
Machine Learning For Robotics David Kelson	28 days	—	EXPIRED	2026-03-29	—	—
Machine Learning For Robotics David Kelson	28 days	—	CANCELLED	2026-03-29	—	—
DevOps Kalen Alex	28 days	—	CANCELLED	2026-03-29	2026-05-01 06:00	—

LibraryMS

Ahmed
i222330@nu.edu.pk
Sign out

REQUESTS

My borrow requests

Track approval status and due dates for all your requests.

BOOK	STATUS	REQUESTED DURATION	REQUESTED AT	DUE DATE	OVERDUE	ACTIONS
Machine Learning For Robotics	CANCELLED	28 days	2026-04-29 08:01	—	—	—
Machine Learning For Robotics	APPROVED	28 days	2026-04-29 07:54	2026-05-27	—	—
Machine Learning For Robotics	APPROVED	28 days	2026-04-29 07:30	2026-05-27	—	—
Machine Learning For Robotics	APPROVED	28 days	2026-04-29 07:05	2026-05-27	—	—
Machine Learning For Robotics	APPROVED	28 days	2026-04-29 06:46	2026-05-27	—	—
Machine Learning For Robotics	APPROVED	28 days	2026-04-29 06:42	2026-05-27	—	—
Machine Learning For Robotics	APPROVED	28 days	2026-04-29 06:36	2026-05-27	—	—
Machine Learning For Robotics	APPROVED	28 days	2026-04-29 06:20	2026-05-27	—	—
Object Oriented Programming	APPROVED	28 days	2026-04-29 06:15	2026-05-27	—	—
Machine Learning For Robotics	APPROVED	28 days	2026-04-29 06:00	2026-05-27	—	—

LibraryMS

Mohammad Rohaan
i222327@nu.edu.pk
Sign out



ADMINISTRATOR

Run Backup + Cleanup

Restore Backup

Welcome to the admin workspace

Mohammad Rohaan (i222327@nu.edu.pk)

- Approve registration requests and manage account lifecycle
- Configure catalog and borrowing policies
- Review operational reports

Review Pending Registrations

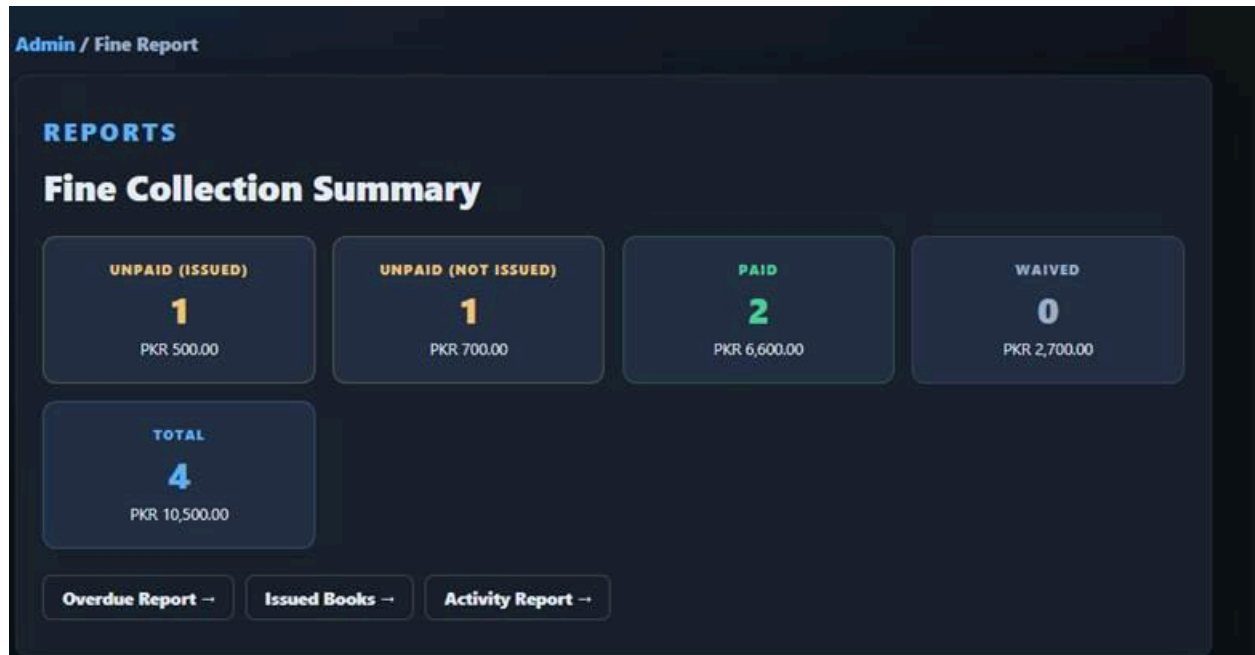
Manage User Accounts

Overdue Report

Issued Books

Fine Report

Activity Report



9. System Architecture

LibraryMS follows a layered MVC architecture enforced by Spring Boot. Each layer has a single responsibility:

Layer	Technology	Responsibility
Presentation	Thymeleaf + Vanilla JS + app.css	Role-specific UI pages; live polling; responsive layout
Controller	Spring MVC Controllers	HTTP routing; request validation; session handling
Service	Spring @Service classes	Business logic; transaction management; rule enforcement
Repository	Spring Data JPA / Hibernate	Data access; pessimistic locking; schema migrations
Database	MySQL 8	Persistent storage of all entities and relationships
Security	Spring Security	Session auth; role-based route protection; CSRF

9.1 Project Structure

- `src/main/java/com/library/controller` — Web controllers (Admin, Librarian, Student)
- `src/main/java/com/library/service` — Business services and transaction logic
- `src/main/java/com/library/repository` — JPA repositories with locking
- `src/main/java/com/library/entity` — Domain entities (Book, BorrowRecord, Fine, Reservation, ...)
- `src/main/java/com/library/security` — Spring Security configuration
- `src/main/resources/templates` — Thymeleaf HTML views per role
- `src/main/resources/static/css/app.css` — Shared responsive stylesheet
- `src/main/resources/static/js` — Live update polling, alert visibility, autofill hardening
- `src/main/resources/application.properties` — Runtime configuration
- `docs/library_db_schema.sql` — Canonical schema file

9.2 Key Configuration

- `server.port=8081` (default, configurable)
- `spring.jpa.hibernate.ddl-auto=update`
- `app.fine.per-day-rate=10.0`
- `app.reservation.pickup-window-hours=96` (4 days)
- Scheduled auto-cancel check configurable via `app.reservation-auto-cancel-check-ms`

10. Testing Activities

Testing was conducted across all three sprints using a hybrid approach combining manual black-box scenario testing, white-box code-path inspection, static diagnostics, and build/startup verification. The testing framework available in the project includes spring-boot-starter-test (JUnit 5, Mockito, AssertJ), spring-security-test, Maven, and IDE diagnostics.

10.1 Test Plan Summary

Testing Type	Technique	Evidence Type
Black-box (Functional)	Manual role-based scenario execution; EP, BVA, EG techniques	Observed outputs, status transitions, UI messages
White-box (Structural)	Service-layer branch/path review; condition and statement coverage	Branch/condition mapping against service methods
Static Quality	IDE Java diagnostics — null-safety warnings	Warning lists before and after fixes
Build Verification	Maven compile and startup runs	Terminal build/start logs — BUILD SUCCESS
Security Behavior	Role-based route testing with Spring Security stack	Observed access control and protected route behavior

10.2 Black-Box Test Cases (All Sprints)

TC ID	Feature	Input / Condition	Expected Result	Sprint	Status
BB-01	Student Registration	Valid student info submitted	Registration pending; appears in admin queue	S1	Pass
BB-02	Admin Approval	Admin approves pending student	Account activated; student can log in	S1	Pass
BB-03	Book Catalog Search	Search by title/author	Correct matching books displayed	S2	Pass
BB-04	Borrow Request	Request book for 7 days	Request status PENDING; appears in librarian queue	S2	Pass
BB-05	Approve Borrow	Librarian approves borrow	Borrow record created; inventory decremented	S2	Pass
BB-06	Book Return	Return on-time active loan	Loan closed; book copy available again	S3	Pass
BB-07	Fine Calculation	Return 2 days late	Fine generated for 2 overdue days	S3	Pass
BB-08	Loan Renewal	Renew active loan (first renewal)	Due date extended; renewal count incremented	S3	Pass

BB-09	Reservation	Reserve unavailable book	Reservation created; FIFO queue position assigned	S3	Pass
BB-10	Admin Reports	Open overdue loans report	Overdue loans displayed accurately	S3	Pass
BB-11	Reservation Auto-Expiry	READY not collected within pickup window	Status automatically transitions to EXPIRED	S3	Pass
BB-12	Fine Report Accuracy	Paid/waived/unpaid with adjustments	Net and waived totals computed correctly	S3	Pass

10.3 White-Box Test Cases (All Sprints)

TC ID	Module	Logic Tested	Coverage	Sprint	Result
WB-01	AccountService	Duplicate email check blocks registration	Branch	S1	Pass
WB-02	LoginService	Role-based login routing to correct dashboard	Branch	S1	Pass
WB-03	BookService	Add/update book stores details correctly	Statement	S2	Pass
WB-04	BorrowService	Active-loan limit enforcement blocks excess requests	Condition	S2	Pass
WB-05	BorrowService	Approve borrow creates record and updates inventory	Branch	S2	Pass
WB-06	ReturnService	Return validates book not already returned	Branch	S3	Pass
WB-07	FineService	returnDate > dueDate triggers correct fine calculation	Condition	S3	Pass
WB-08	FineManagementService	Mark fine paid/waived updates status correctly	Branch	S3	Pass
WB-09	RenewalService	All renewal blocking conditions enforced correctly	Condition	S3	Pass
WB-10	ReservationService	READY reservation transitions to EXPIRED after window	Path	S3	Pass

10.4 Defect / Bug Report Summary

Bug ID	Description	Severity	Status	Resolution
BUG-01	App startup failed dropping reservation unique index (FK dependency)	High	Fixed	Safe migration fallback + pre-indexing strategy
BUG-02	Reservation cancel/fulfill conflict errors (HTTP 409)	High	Fixed	Conflict-safe repository operations + transaction flow refinement
BUG-03	Student reservation page intermittent server error	High	Fixed	Adjusted transactional behavior in list/reconcile flow
BUG-04	Queue position duplication risk in reservation	Medium	Fixed	Active queue resequencing with pessimistic locking
BUG-05	Live updates interrupting focused controls	Medium	Fixed	Interaction-aware polling pause logic implemented
BUG-06	Fine report waived/net inconsistencies	Medium	Fixed	Report and fine aggregation logic corrected
BUG-07	Duplicate return could corrupt inventory count	High	Fixed	Already-returned validation guard added
BUG-08	Renewal visible after maximum limit reached	Medium	Fixed	Limit enforcement tightened in RenewalService
BUG-09	IDE null-safety warnings in reservation modules	Low	Fixed	Non-null guards added; warnings cleared

10.5 Test Execution Results

Test Type	Total Cases	Passed
Black-Box (Functional)	12	12
White-Box (Structural)	10	10
Total	22	22

All critical defects were resolved prior to final submission. No blocking issues remain.

11. Team and Work Division

11.1 Ali Bin Salman — 22i-0894

Role: Project Manager, Scrum Master, Developer

- Led overall project planning and sprint coordination.

- Managed timeline, sprint goals, and delivery schedule.
- Conducted requirements gathering and analysis across all sprints.
- Designed the database schema for book and borrowing modules (Sprint 2).
- Implemented core backend APIs for book management.
- Managed and maintained the Trello Scrum board across all three sprints.
- Conducted daily stand-ups and monitored sprint progress as Scrum Master.
- Ensured Scrum practices were correctly followed.
- Contributed to Account Management module backend logic.

11.2 Muhammad Rohaan — 22i-2327

Role: Developer, Requirements Analyst, Architect

- Designed the overall layered system architecture.
- Defined the database entity relationships and schema structure.
- Ensured proper role-based access control implementation throughout.
- Implemented core backend functionality and service-layer logic.
- Implemented borrowing workflow logic and approval/rejection system (Sprint 2).
- Prepared user stories, structured specifications, and backlog items.
- Supervised system architecture and cross-module integration.
- Developed book management, search, fine, reservation, and report interfaces.

11.3 Muhammad Atyab — 22i-1191

Role: Tester, UI Designer, Developer

- Designed user interface layouts for all portals and modules.
- Implemented frontend components for Sprint 1, 2, and Sprint 3 features.
- Integrated frontend pages with backend Spring MVC endpoints.
- Performed functional black-box and white-box testing across all sprints.
- Identified and reported bugs during sprint execution.
- Verified role-based login and all approval workflows.
- Assisted in debugging, refining features, and documentation preparation.

11.4 Shared Responsibilities (All Members)

- Requirements analysis and discussion.
- Sprint planning, task breakdown, and backlog refinement.
- Participation in Scrum ceremonies (stand-ups, retrospectives).
- Cross-review of completed features before marking tasks Done.
- Documentation preparation and final submission readiness.

12. Conclusion

The Library Management System (LibraryMS) was successfully designed, developed, and delivered across three Scrum sprints by Team Stack Architects. The final system is a complete, integrated, multi-role web application that handles every stage of the library lifecycle — from student registration and book catalog management through borrowing, returning, fine calculation, loan renewals, reservation queuing, and administrative reporting.

Sprint 1 laid the foundation with secure account management and role-based authentication. Sprint 2 brought the core operational capabilities — book catalog management, search, and the borrowing workflow. Sprint 3 completed the system by delivering the advanced modules: automated fines, loan renewals, a FIFO reservation queue with automatic expiry, full fine management, and live-updating admin reports.

Throughout development, the team applied black-box and white-box testing techniques, maintained a Scrum board, tracked velocity through burn-down charts, and resolved all critical defects before final submission. The system satisfies all functional requirements defined in the original product backlog and meets the non-functional requirements for performance, security, usability, reliability, scalability, and maintainability.

Recommended Future Enhancements

- Add automated integration tests for reservation expiry and queue ordering.
- Introduce a schema migration versioning framework (Flyway or Liquibase).
- Add environment-specific configuration profiles and secrets management.
- Establish a CI/CD pipeline covering compile, test, lint, and schema consistency checks.
- Enable HTTPS, secure cookie settings, and external credential management for production.
- Extend the student portal with email notifications for reservation readiness and overdue reminders.